HTTP Server

---

1. HTTP Server Overview

HTTP (HyperText Transfer Protocol) is the most widely used application layer protocol on the Internet. It is based on TCP and uses a **request-response** model: the client sends a GET request, and the server returns a status code + data.

The HTTP server runs on the ESP32-S3 and provides web page browsing and API interfaces. This experiment uses the `WebServer` library for route registration: `server.on(path, callback)` maps URL paths to handler functions, and `server.handleClient()` processes the request queue in `loop()`. Access `http://<ESP32_IP>/` in a browser to see the HTML page, and access `/get?msg=hello` to return an API string.

Typical Application Scenarios:

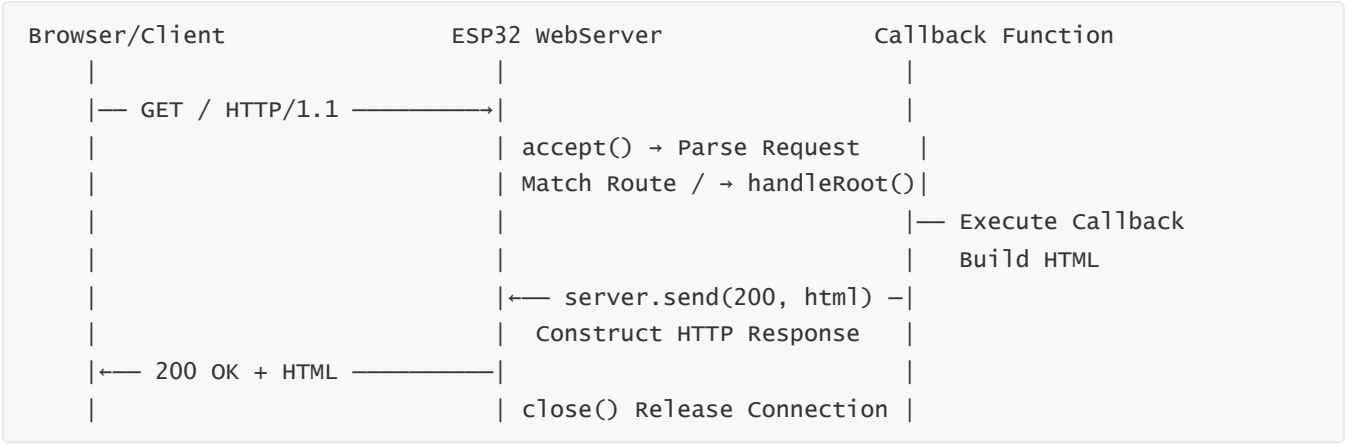
Application Type	Example
Device Management Panel	Web page to configure Wi-Fi, view sensor data
Simple API	Mobile App controls GPIO via HTTP GET
File Upload	POST uploads configuration file to SPIFFS

---

2. Core Concepts

2.1 WebServer Routing Model

The complete processing chain of a single HTTP request inside ESP32:



`server.handleClient()` is called frequently in `loop()`, processing one queued request at a time (non-blocking).

```
server.on(path, handler)    // Register route: map URL path to callback function
server.on("/", handleRoot)  // GET / → handleRoot()
server.on("/get", handleGet)// GET /get?msg=xxx → handleGet()
server.begin()              // socket() → bind(80) → listen()
server.handleClient()       // Process next request in queue (non-blocking)
```

Method	Purpose	Blocking?
`server.on(path, callback)`	Register route, call callback when path matches	—
`server.begin()`	Bind port 80 and start listening	No
`server.handleClient()`	Process next HTTP request in queue	No (one at a time)
`server.send(code, type, body)`	Construct and send complete HTTP response	No
`server.arg("key")`	Read URL query parameter `?key=value`	No

2.2 HTTP Response Construction

`server.send(code, mime, body)` internally constructs a standard HTTP response message:

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 123\r\n
\r\n
<html><body>...</body></html>
```

Developers only need to care about three parameters:

Parameter	Meaning	Example
`code`	HTTP status code	`200`, `404`, `500`
`mime`	Content type (MIME)	`"text/html"`, `"text/plain"`, `"application/json"`
`body`	Response body string	HTML source, JSON string, plain text

---

3. Hardware Dependencies

Pure software experiment. Must first connect to router via WIFI-STA.

---

4. Project Architecture and Flow

```
setup()
  ├─ connectWiFi()
  ├─ server.on("/", handleRoot)        // Register routes
  ├─ server.on("/get", handleGet)
  └─ server.begin()                    // Start listening

loop()
  └─ server.handleClient()             // Process request queue
```

---

5. Source Code Analysis

Complete source code: `Source code → Arduino → 3.Network Communication → HTTP_Server → HTTP_Server.ino`

5.1 Home Page Route Handler

```
#include <WebServer.h>
WebServer server(80);    // Listen on port 80 (HTTP default port)

void handleRoot() {
  String html = "<html><body>";
  html += "<h1>ESP32-S3 HTTP Server</h1>";
  html += "<p>Hello from ESP32</p>";
  html += "</body></html>";
  server.send(200, "text/html", html);
}
```

5.2 GET API Route Handler

Extract the value of `msg` from URL query parameters and echo it back.

```
void handleGet() {
  String msg = server.arg("msg");    // Get ?msg=xxx parameter
  server.send(200, "text/plain", "GET OK: " + msg);
}
```

> `/get?msg=hello` → `server.arg("msg")` returns `"hello"`. Returns empty string when parameter name does not exist.

5.3 `setup()` and `loop()`

```
void setup() {
  Serial.begin(115200);
  connectWiFi();

  server.on("/", handleRoot);
  server.on("/get", handleGet);
  server.begin();

  Serial.printf("HTTP Server started | http://%s/\n", WiFi.localIP().toString().c_str());
}

void loop() {
  server.handleClient();   // Process one HTTP request (non-blocking)
  delay(1);
}
```

> `handleClient()` processes one client per call (multiple clients queued). Must be called frequently in `loop()`, otherwise clients will experience "stuttering".

---

6. Operation Flow

6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

1. Build and flash
2. Access the IP address printed on serial from PC or mobile browser

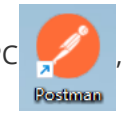
6.2 Operation Steps

Step 1: Flash program, record server IP

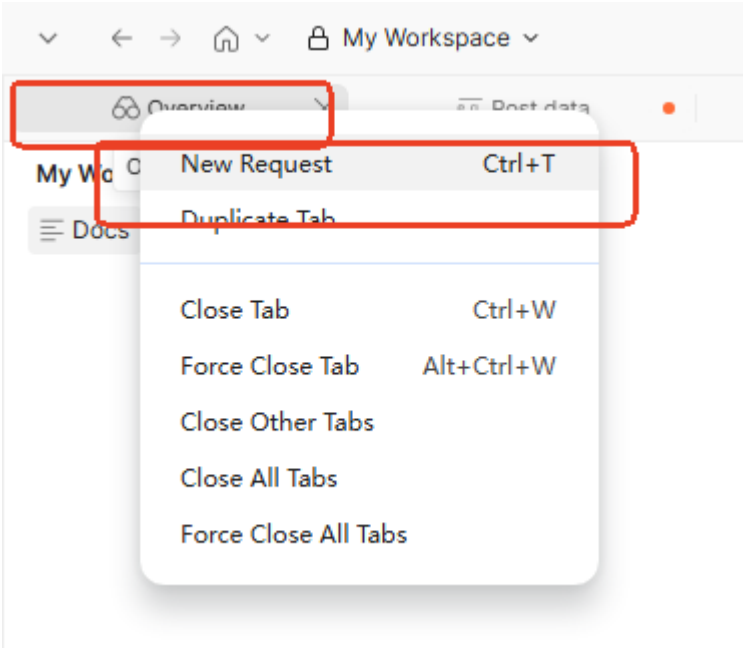
Build and flash to ESP32-S3 development board. Open serial monitor (baud rate 115200), wait for WiFi connection success, and note down the server address printed on serial:

```
================================
 HTTP Server Startup successful!
 Access: http://192.168.11.229:80/
================================
```

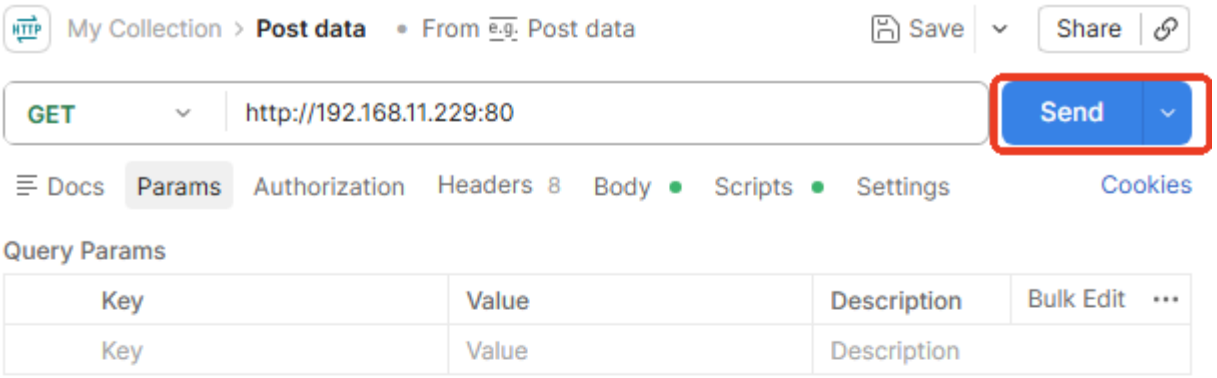
Step 2: Access home page with Postman

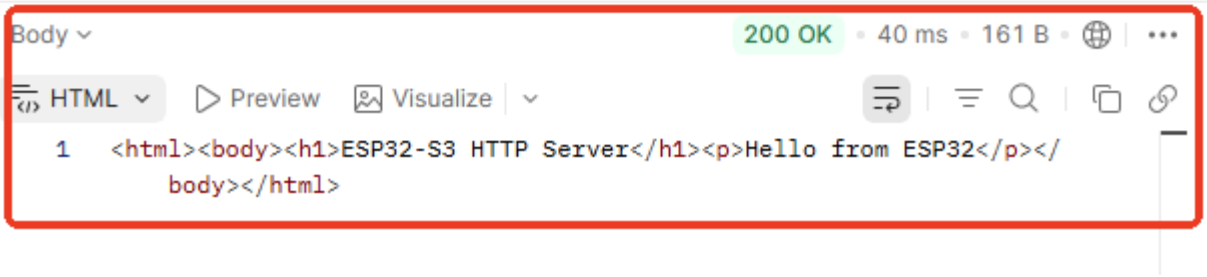
Log into Postman on PC 

, create a new request, enter the local address http://192.168.11.229:80/



Request result as shown

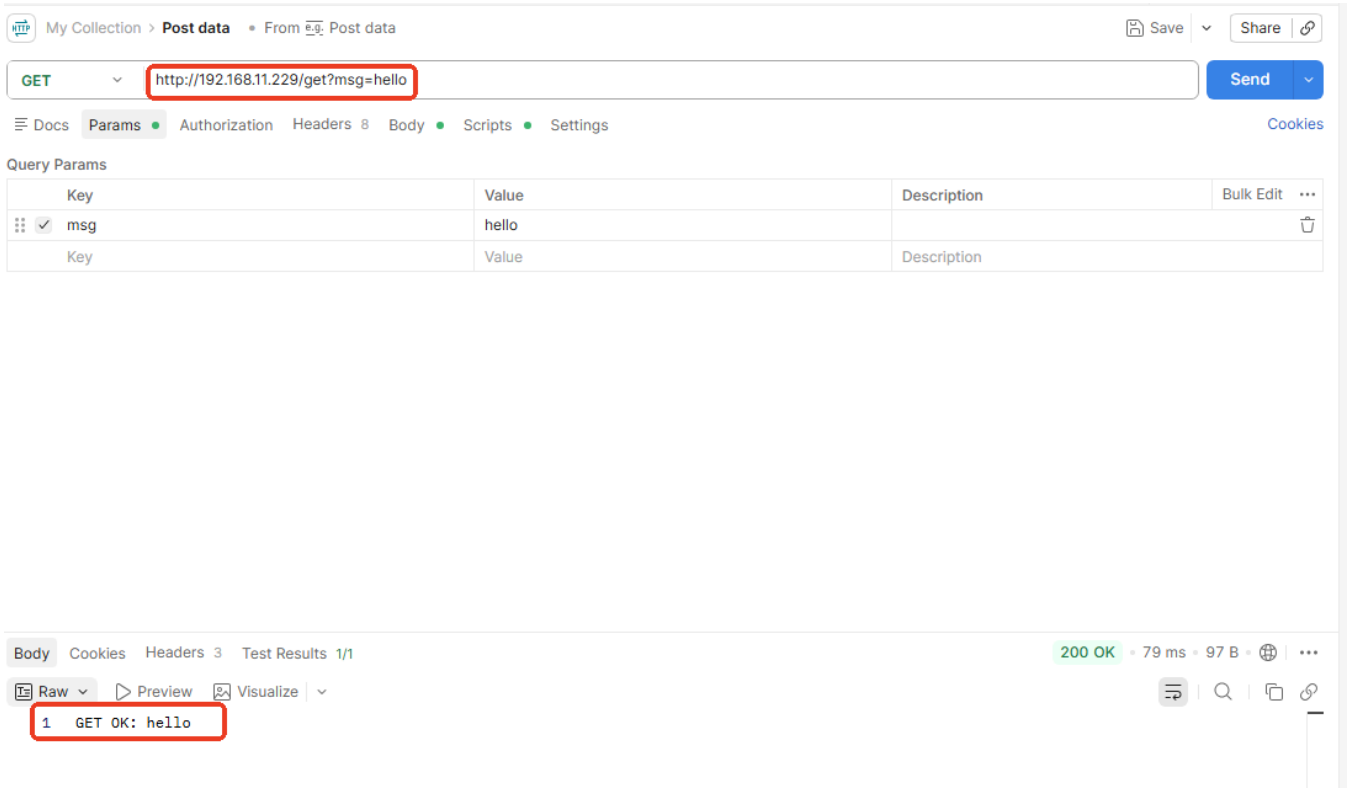




> **ESP32-S3 HTTP Server**
> Hello from ESP32

Step 3: Test API interface (with parameters)

Enter `http://192.168.11.229/get?msg=hello` in the browser address bar (replace IP with actual address), press Enter. Browser displays:

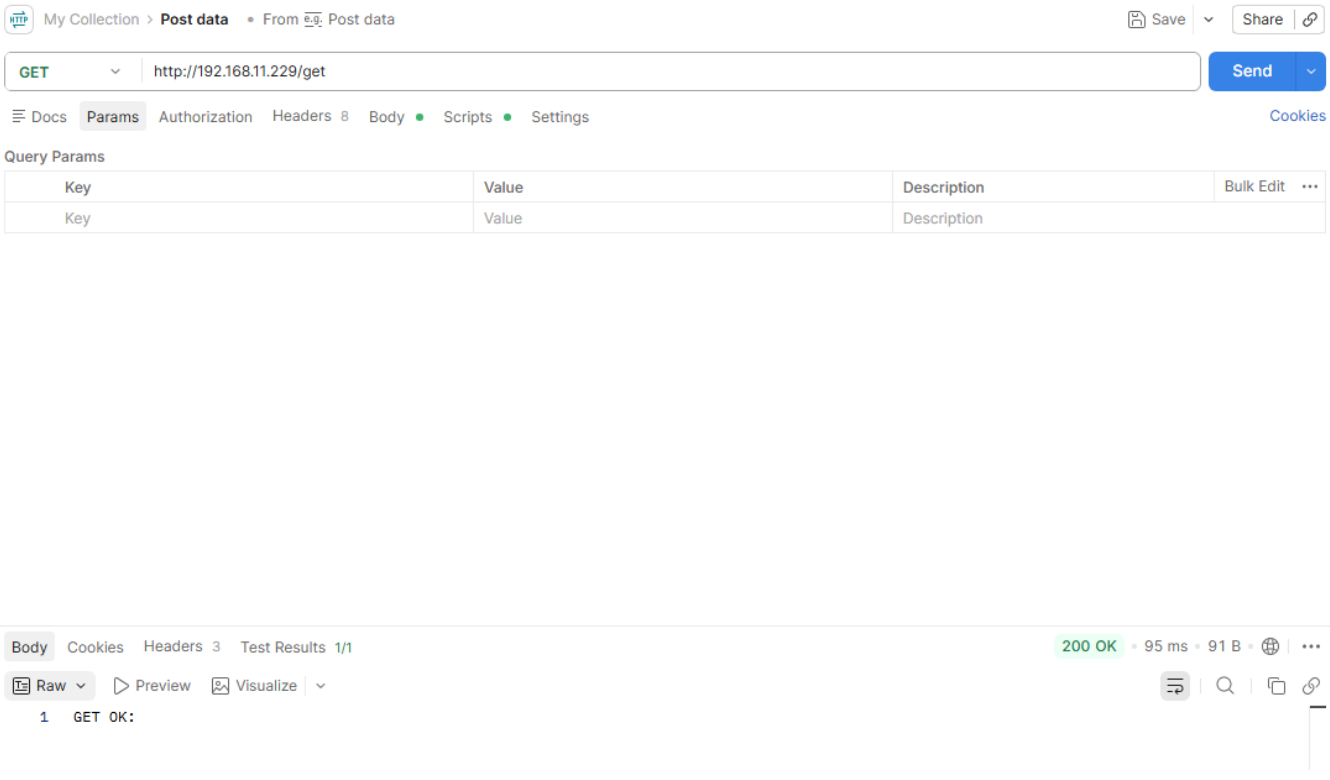


> GET OK: hello

Change the parameter after `msg=` to any string (such as `?msg=world`), and the page will display the corresponding value.

Step 4: Test API interface (without parameters)

Enter `http://192.168.11.229/get` (without parameters) in the browser address bar, press Enter. Browser displays:



> GET OK:

The `msg` parameter is empty, indicating that `server.arg("msg")` returns an empty string when no parameter is passed.

6.3 Serial Monitor Output

6.4 Verification Methods

Operation	Expected Result
Browser access home page	Display HTML page
API access with parameters	Correctly return parameter value
Open multiple browsers simultaneously	Queued processing, responses one by one

---

7. Common Issues

Symptom	Cause	Solution
No response when accessing via browser	`handleClient()` not called frequently enough	Remove long `delay()` in `loop()`
`server.arg("msg")` returns empty	Parameter name misspelled or not passed in URL	Check URL format `?msg=xxx`
Chinese characters garbled on page	Content-Type charset not specified	`server.send(200, "text/html; charset=utf-8", html)`